2. Funkcje. Zmienne globalne i lokalne

1. Funkcja jako podprogram

Funkcja w C++ to nazwany podprogram, który wykonuje określone zadanie. Może zwracać wynik albo jedynie wykonywać działanie. Dzięki funkcjom program można podzielić na mniejsze części, co ułatwia projektowanie, testowanie i modyfikację kodu.

> Funkcja powinna mieć jasną odpowiedzialność. Jeżeli jedna funkcja robi zbyt wiele rzeczy naraz, trudniej ją sprawdzić i ponownie wykorzystać.

2. Deklaracja i definicja funkcji

Deklaracja, nazywana też prototypem lub nagłówkiem funkcji, informuje kompilator o typie zwracanej wartości, nazwie funkcji oraz parametrach. Definicja zawiera ciało funkcji, czyli instrukcje wykonywane po jej wywołaniu.

```
// deklaracja
double poleProstokata(double bok1, double bok2);

// definicja
double poleProstokata(double bok1, double bok2) {
    return bok1 * bok2;
}
```

3. Wywołanie funkcji

Wywołanie funkcji powoduje wykonanie instrukcji zapisanych w jej definicji. W nawiasach podaje się argumenty aktualne, czyli konkretne wartości lub zmienne przekazywane do funkcji.

```
double a = 3;
double b = 5;
double wynik = poleProstokata(a, b);
```

4. Parametry formalne i argumenty aktualne

Pojęcie	Znaczenie
parametr formalny	Zmienna zapisana w deklaracji lub definicji funkcji. Określa, jakich danych funkcja oczekuje.
argument aktualny	Konkretna wartość, zmienna lub wyrażenie podane podczas wywołania funkcji.

5. Sposoby przekazywania danych

Przez wartość

Funkcja otrzymuje kopię argumentu. Zmiana parametru wewnątrz funkcji nie wpływa na zmienną przekazaną przy wywołaniu.

```
int podwoj(int x) {
    x = 2 * x;
    return x;
}
```

Przez referencję

Funkcja pracuje na oryginalnej zmiennej. Można w ten sposób zwrócić wynik przez parametr wyjściowy.

```
void zwieksz(int &x) {
    x++;
}
```

Przez referencję do const

Ten sposób pozwala uniknąć kopiowania dużych danych i jednocześnie uniemożliwia zmianę argumentu w funkcji.

```
double poleProstokata(const double &a, const double &b) {
    return a * b;
}
```

6. Zmienne globalne

Zmienna globalna jest zadeklarowana poza funkcjami. Może być widoczna w wielu funkcjach, zwykle od miejsca deklaracji do końca pliku. Jej czas życia trwa od uruchomienia programu do jego zakończenia.

> Nadużywanie zmiennych globalnych utrudnia analizę programu, ponieważ wiele funkcji może zmieniać te same dane. W ćwiczeniach warto rozumieć ten mechanizm, ale w projektowaniu programu lepiej przekazywać dane przez parametry.

7. Zmienne lokalne

Zmienna lokalna jest zadeklarowana w funkcji lub w bloku kodu. Jej zakres widoczności jest ograniczony do tej funkcji albo bloku. Po opuszczeniu zakresu zmienna lokalna przestaje istnieć.

```
int main() {
    int x = 5;
    if (x > 0) {
        int y = 10; // y widoczne tylko w bloku if
    }
    // tutaj y nie jest dostępne
}
```

8. Przesłanianie zmiennych

Przesłanianie występuje wtedy, gdy w zakresie lokalnym zdefiniowano nazwę taką samą jak nazwa zmiennej z zakresu zewnętrznego. W takim miejscu używana jest zmienna lokalna, a nie zewnętrzna.

```
int liczba = 100;

int main() {
    int liczba = 7;
    cout << liczba; // 7
}
```